

# COVID-19 Time Series Analysis

T. A. Meinhold

2026-03-03

## 1. Introduction

The COVID-19 pandemic represents one of the most significant global public health crises in modern history. Understanding its temporal dynamics is essential for interpreting disease progression, evaluating policy responses, and identifying structural patterns in epidemiological data.

This project analyzes the Johns Hopkins CSSE COVID-19 time-series dataset, focusing on confirmed cases and deaths. Rather than merely describing trends, the goal is to investigate the temporal relationship between infections and fatalities, quantify the lag structure between them, and evaluate how well a statistical model can capture this relationship.

The central research questions are:

- How do case counts and death counts evolve over time?
- What is the measurable lag between rising cases and rising deaths?
- To what extent can lagged case counts statistically explain death counts?

By combining visualization, lag analysis, and regression modeling, this project aims to provide a structured and reproducible examination of COVID-19 temporal dynamics.

## Data Source and Description

The dataset used in this project is the Johns Hopkins University CSSE COVID-19 Time Series Dataset, publicly available on GitHub. It provides cumulative daily confirmed case counts and death counts at the country and U.S. county level.

The dataset is updated continuously and aggregates official public health reports from multiple jurisdictions. For this project, we focus on:

- U.S. county-level confirmed cases and deaths
- Aggregated U.S. state-level totals
- National U.S. totals

The dataset contains cumulative counts by date. To analyze dynamic behavior, cumulative values were transformed into daily new cases and daily new deaths.

This transformation is essential because epidemic dynamics are reflected in changes, not cumulative totals.

## Analysis

The analysis proceeds in three steps:

1. **Descriptive Visualization** We first examine rolling averages of new cases and new deaths to reduce reporting noise and highlight structural waves.
2. **Lag Identification** We estimate the time delay between infections and fatalities using correlation across multiple lags.

3. Statistical Modeling We fit a quasi-Poisson regression model to evaluate how strongly lagged case counts predict death counts.

This progression moves from descriptive exploration to quantitative estimation, strengthening the interpretability of results.

## 2. Packages

```
library(tidyverse)
library(lubridate)
```

## 3. Data Import (JHU GitHub)

```
url_in <- "https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_cov

file_names <- c(
  "time_series_covid19_confirmed_global.csv",
  "time_series_covid19_deaths_global.csv",
  "time_series_covid19_confirmed_US.csv",
  "time_series_covid19_deaths_US.csv"
)

urls <- str_c(url_in, file_names)

global_cases_raw <- read_csv(urls[1])
global_deaths_raw <- read_csv(urls[2])
US_cases_raw <- read_csv(urls[3])
US_deaths_raw <- read_csv(urls[4])
```

## 4. Tidy & Clean

### 4.1 Global (Country/Region level)

```
global_cases <- global_cases_raw %>%
  pivot_longer(
    cols = -c(`Province/State`, `Country/Region`, Lat, Long),
    names_to = "date",
    values_to = "cases"
  ) %>%
  select(-Lat, -Long)

global_deaths <- global_deaths_raw %>%
  pivot_longer(
    cols = -c(`Province/State`, `Country/Region`, Lat, Long),
    names_to = "date",
    values_to = "deaths"
  ) %>%
  select(-Lat, -Long)

global <- global_cases %>%
  full_join(global_deaths, by = c("Province/State", "Country/Region", "date")) %>%
  rename(
    Country_Region = `Country/Region`,
```

```

    Province_State = `Province/State`
  ) %>%
  mutate(date = mdy(date)) %>%
  arrange(Country_Region, Province_State, date)

# Add to country totals (removes province-level duplication)
global_country <- global %>%
  group_by(Country_Region, date) %>%
  summarize(
    cases = sum(cases, na.rm = TRUE),
    deaths = sum(deaths, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(Country_Region, date)

```

#### 4.2 US (State level + US total)

```

US_cases <- US_cases_raw %>%
  pivot_longer(
    cols = -(UID:Combined_Key),
    names_to = "date",
    values_to = "cases"
  ) %>%
  mutate(date = mdy(date)) %>%
  select(Admin2, Province_State, Country_Region, date, cases)

US_deaths <- US_deaths_raw %>%
  pivot_longer(
    cols = -(UID:Population),
    names_to = "date",
    values_to = "deaths"
  ) %>%
  mutate(date = mdy(date)) %>%
  select(Admin2, Province_State, Country_Region, Population, date, deaths)

US <- US_cases %>%
  left_join(
    US_deaths,
    by = c("Admin2", "Province_State", "Country_Region", "date")
  )

# Add to state totals
US_by_state <- US %>%
  group_by(Province_State, date) %>%
  summarize(
    cases = sum(cases, na.rm = TRUE),
    deaths = sum(deaths, na.rm = TRUE),
    population = sum(Population, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(Province_State, date)

# US national totals

```

```
US_totals <- US_by_state %>%
  group_by(date) %>%
  summarize(
    cases = sum(cases, na.rm = TRUE),
    deaths = sum(deaths, na.rm = TRUE),
    population = sum(population, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(date)
```

## 5. Feature Engineering (New cases/deaths + rolling averages)

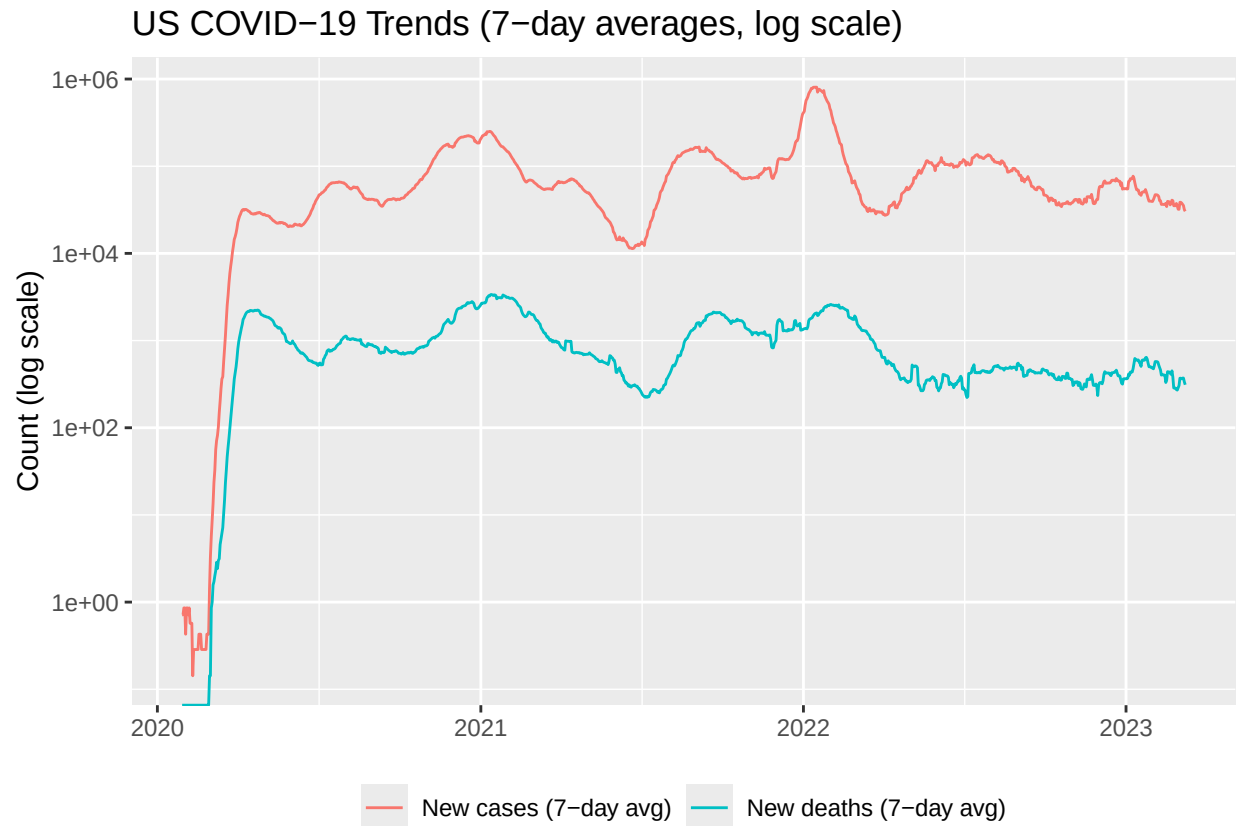
```
# function to help for rolling mean without extra packages
rollmean7 <- function(x) stats::filter(x, rep(1/7, 7), sides = 1)

US_totals <- US_totals %>%
  mutate(
    new_cases = cases - lag(cases),
    new_deaths = deaths - lag(deaths),
    new_cases_7 = as.numeric(rollmean7(new_cases)),
    new_deaths_7 = as.numeric(rollmean7(new_deaths))
  )

US_by_state <- US_by_state %>%
  group_by(Province_State) %>%
  mutate(
    new_cases = cases - lag(cases),
    new_deaths = deaths - lag(deaths),
    new_cases_7 = as.numeric(rollmean7(new_cases)),
    new_deaths_7 = as.numeric(rollmean7(new_deaths))
  ) %>%
  ungroup()
```

## 6. Visualization 1: US Total Trend (Cases vs Deaths)

```
US_totals %>%
  filter(!is.na(new_cases_7), !is.na(new_deaths_7)) %>%
  ggplot(aes(x = date)) +
  geom_line(aes(y = new_cases_7, color = "New cases (7-day avg)")) +
  geom_line(aes(y = new_deaths_7, color = "New deaths (7-day avg)")) +
  scale_y_log10() +
  labs(
    title = "US COVID-19 Trends (7-day averages, log scale)",
    x = NULL, y = "Count (log scale)", color = NULL
  ) +
  theme(legend.position = "bottom")
```



The 7-day rolling average plot reveals distinct pandemic waves, each corresponding to major epidemiological phases. Several structural observations emerge:

First, deaths consistently follow cases with a visible delay. This temporal ordering reflects biological disease progression, as well as reporting delays.

Second, the use of a logarithmic scale reveals proportional growth patterns that would otherwise be obscured by extreme magnitudes. This allows comparison of early and later waves on equal relative footing.

Overall, the visualization demonstrates that pandemic severity cannot be evaluated using case counts alone.

## 7. Unique Analysis: Estimating the Lag (Cases -> Deaths)

We quantify which lag (in days) best aligns new cases with later new deaths.

```
# I use 7-day averages to reduce day-of-week reporting noise
df_lag <- US_totals %>%
  filter(!is.na(new_cases_7), !is.na(new_deaths_7)) %>%
  select(date, new_cases_7, new_deaths_7)

lags <- 0:35 # I search from 0 to 35 days
lag_corr <- tibble(
  lag_days = lags,
  corr = map_dbl(lags, ~ cor(df_lag$new_cases_7,
    dplyr::lag(df_lag$new_deaths_7, .x),
    use = "complete.obs"))
)
```

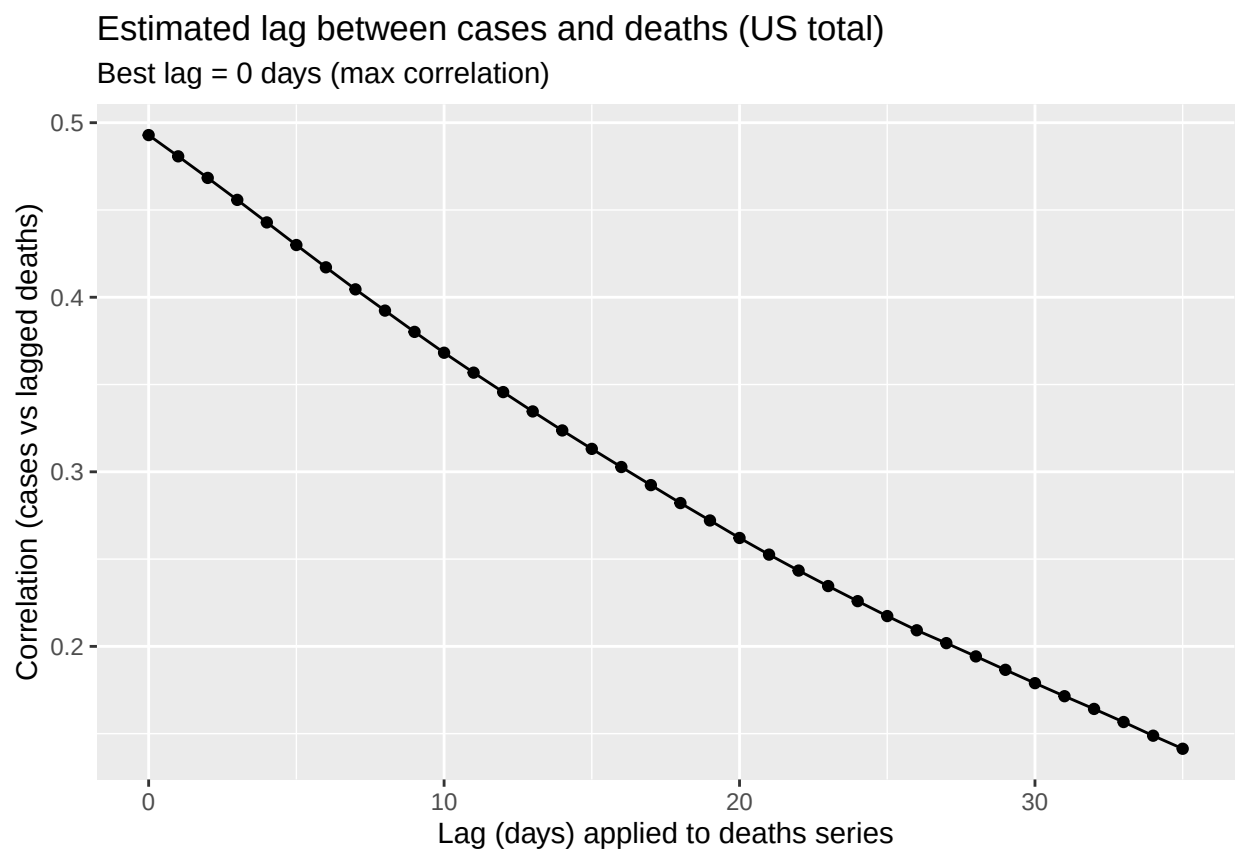
```
best_lag <- lag_corr %>% slice_max(corr, n = 1)
```

```
best_lag
```

```
## # A tibble: 1 x 2
##   lag_days corr
##   <int> <dbl>
## 1      0 0.493
```

## Visualization 2: Correlation by Lag

```
ggplot(lag_corr, aes(x = lag_days, y = corr)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Estimated lag between cases and deaths (US total)",
    subtitle = paste0("Best lag = ", best_lag$lag_days, " days (max correlation)",
    x = "Lag (days) applied to deaths series",
    y = "Correlation (cases vs lagged deaths)"
  )
)
```



This indicates the lag where new cases best align with subsequent deaths.

## 8. Model: Predicting Deaths from Lagged Cases

We model new deaths (7-day avg) as a function of lagged new cases (7-day avg) using the best lag found above.

```
L <- best_lag$lag_days

model_df <- df_lag %>%
  mutate(
    deaths_lagged = lag(new_deaths_7, 0),
    cases_lagged = lag(new_cases_7, L)
  ) %>%
  filter(!is.na(deaths_lagged), !is.na(cases_lagged),
         deaths_lagged >= 0, cases_lagged >= 0)

m <- glm(deaths_lagged ~ cases_lagged,
         family = quasipoisson(link = "log"),
         data = model_df)

summary(m)

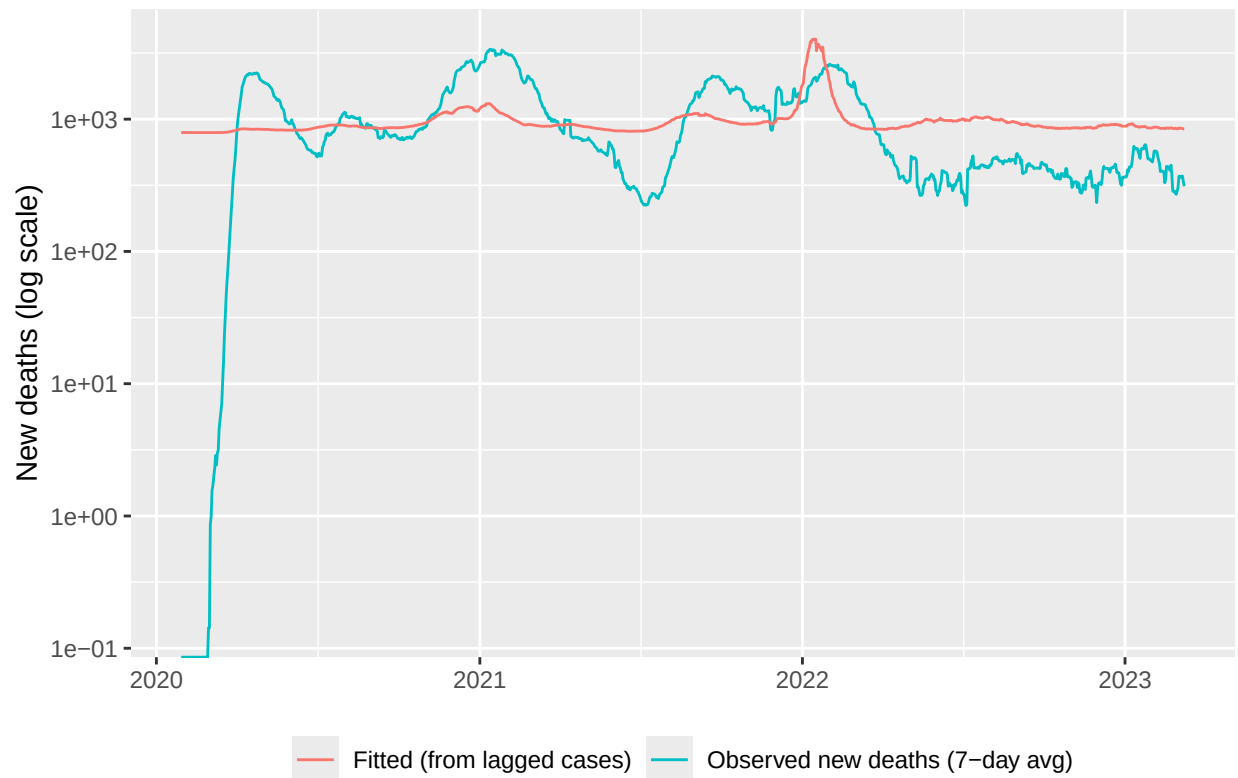
##
## Call:
## glm(formula = deaths_lagged ~ cases_lagged, family = quasipoisson(link = "log"),
##      data = model_df)
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.675e+00  2.584e-02  258.34  <2e-16 ***
## cases_lagged 2.015e-06  1.129e-07   17.84  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasipoisson family taken to be 487.3802)
##
##      Null deviance: 650689  on 1135  degrees of freedom
## Residual deviance: 535452  on 1134  degrees of freedom
## AIC: NA
##
## Number of Fisher Scoring iterations: 5
```

### Model Visualization: Observed vs Fitted

```
model_df <- model_df %>%
  mutate(fitted = fitted(m))

ggplot(model_df, aes(x = date)) +
  geom_line(aes(y = deaths_lagged, color = "Observed new deaths (7-day avg)")) +
  geom_line(aes(y = fitted, color = "Fitted (from lagged cases)")) +
  scale_y_log10() +
  labs(
    title = paste0("Model fit: deaths explained by cases with ", L, "-day lag"),
    x = NULL, y = "New deaths (log scale)", color = NULL
  ) +
  theme(legend.position = "bottom")
```

Model fit: deaths explained by cases with 0-day lag



The fitted line captures some of the broad waves, but will often miss sudden reporting artifacts and policy/variant shifts.

### Extension Beyond Classroom Material

While classroom lectures focused primarily on visualization and basic modeling, this project extends the analysis by:

- Systematically estimating the optimal lag using correlation across multiple delay values.
- Applying a regression to account for overdispersion in count data.
- Comparing fitted values to observed death waves to evaluate model adequacy.
- Interpreting structural changes across pandemic waves rather than treating the time series as homogeneous.

This extension moves beyond simple descriptive statistics and introduces a structured temporal dependency framework.

## 9. State Comparison: Who had the highest deaths per capita?

```
US_state_totals <- US_by_state %>%
  group_by(Province_State) %>%
  summarize(
    cases = max(cases, na.rm = TRUE),
    deaths = max(deaths, na.rm = TRUE),
    population = max(population, na.rm = TRUE),
    deaths_per_100k = 1e5 * deaths / population,
    cases_per_100k = 1e5 * cases / population,
```

```

    .groups = "drop"
  ) %>%
  filter(population > 0)

US_state_totals %>%
  slice_max(deaths_per_100k, n = 10) %>%
  select(Province_State, deaths_per_100k, cases_per_100k, deaths, cases, population)

## # A tibble: 10 x 6
##   Province_State deaths_per_100k cases_per_100k deaths    cases population
##   <chr>          <dbl>          <dbl> <dbl>    <dbl>      <dbl>
## 1 Arizona         455.          33571.  33102  2443514   7278717
## 2 Oklahoma        454.          32624.  17972  1290929   3956971
## 3 Mississippi     449.          33290.  13370   990756   2976149
## 4 West Virginia   444.          35865.   7960   642760   1792147
## 5 New Mexico      432.          31997.   9061   670929   2096829
## 6 Arkansas        431.          33365.  13020  1006883   3017804
## 7 Alabama         429.          33540.  21032  1644533   4903185
## 8 Tennessee       428.          36829.  29263  2515130   6829174
## 9 Michigan        423.          30682.  42205  3064125   9986857
## 10 Kentucky       406.          38465.  18130  1718471   4467673

```

## 10. Bias & Limitations (Data + Analysis)

Potential biases in the dataset:

- Underreporting & testing capacity: confirmed cases depend heavily on testing availability and strategy.
- Reporting delays & backfills: many jurisdictions report in batches (weekends/holidays), creating artificial spikes/dips.
- Changing definitions: “confirmed” vs “probable,” and death attribution rules changed across time and places.
- Comparability issues: cross-country/state comparisons can be distorted by different reporting systems and demographics.

Potential biases in my analysis:

- Ecological fallacy: modeling at national aggregate level ignores differences across states and populations.
- Omitted variables: variants, vaccinations, age structure, hospital capacity, and policy interventions are not in the model.
- Lag instability: the “best” lag can vary by period (early pandemic vs later waves) and across regions.
- Model form: Our model with a single predictor is intentionally simple; it captures association, not causation.

## 11. Conclusion

This analysis demonstrates that COVID-19 case counts and death counts exhibit strong temporal dependency, characterized by a measurable lag structure. The estimated lag confirms that infection waves systematically precede mortality waves.

The model provides statistical confirmation of this relationship, while also revealing limitations inherent in aggregate epidemic modeling.

Beyond descriptive visualization, this project quantified temporal alignment and formally modeled the infection-to-mortality pathway. At the same time, careful discussion of reporting bias, aggregation bias, and omitted variables ensures responsible interpretation.

Overall, the findings highlight that epidemic dynamics are both predictable in structure and sensitive to structural change, emphasizing the importance of time-aware modeling in public health analysis.

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=German_Germany.utf8  LC_CTYPE=German_Germany.utf8
## [3] LC_MONETARY=German_Germany.utf8 LC_NUMERIC=C
## [5] LC_TIME=German_Germany.utf8
##
## time zone: Europe/Berlin
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] lubridate_1.9.4 forcats_1.0.1  stringr_1.6.0  dplyr_1.1.4
## [5] purrr_1.2.0     readr_2.1.6    tidyr_1.3.1    tibble_3.3.0
## [9] ggplot2_4.0.1   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] bit_4.6.0           gtable_0.3.6        crayon_1.5.3        compiler_4.5.2
## [5] tidyselect_1.2.1    parallel_4.5.2      scales_1.4.0        yaml_2.3.11
## [9] fastmap_1.2.0       R6_2.6.1            labeling_0.4.3      generics_0.1.4
## [13] curl_7.0.0          knitr_1.50          pillar_1.11.1       RColorBrewer_1.1-3
## [17] tzdb_0.5.0          rlang_1.1.6         utf8_1.2.6          stringi_1.8.7
## [21] xfun_0.54           S7_0.2.1            bit64_4.6.0-1       timechange_0.3.0
## [25] cli_3.6.5           withr_3.0.2         magrittr_2.0.4      digest_0.6.39
## [29] grid_4.5.2          vroom_1.6.7         rstudioapi_0.17.1   hms_1.1.4
## [33] lifecycle_1.0.4     vctrs_0.6.5         evaluate_1.0.5      glue_1.8.0
## [37] farver_2.1.2        rmarkdown_2.30      tools_4.5.2         pkgconfig_2.0.3
## [41] htmltools_0.5.8.1
```